



AegisTrace Technical Architecture

Reference Implementation of the AI-IDP Standard

Pierre-Edward Procyk
Founder / CEO
Cognitive Industries — Les Industries Cognitives
Saguenay, Québec, Canada
p.procyk.media@gmail.com

Document ID: AEGISTRACE-ARCH-2.0.0
Version: 2.0.0
Date: 2026-08-01
Classification: Public

1. Overview

AegisTrace is the reference implementation of the AI-IDP standard. It demonstrates end-to-end persistent AI Actor identity, delegation, provenance, traceability, quality evidence, accountability, and permanent audit. The architecture is provider-neutral, jurisdiction-aware (Canadian primary), and built on established cryptographic primitives. This document describes the architecture's layers, components, data flows, security properties, and operational characteristics.

The architecture is grounded in seven core principles: (1) the persistent agent is the identifiable logical software actor, surviving all execution-context changes; (2) the agent instance is a specific execution tied to a concrete provider, model, version, and deployment; (3) the model is an execution component, not an agent; (4) the provider supplies or operates a model or service; (5) the principal grants authority; (6) the controller remains legally accountable; and (7) the event record connects all of them through a hash-chained, signed, append-only ledger.

AegisTrace is implemented in Python 3.12 with strict static typing, Ed25519 signatures via the cryptography library, JSON Schema validation via jsonschema, FastAPI for the HTTP API, and SQLite for local canonical storage (with a PostgreSQL-compatible production design). The implementation is testable, reproducible, and deployable in containerized environments.

2. Layered Architecture

The architecture is organized into six layers, each with a clear responsibility and well-defined interfaces to adjacent layers.

2.1 Identity Layer

The identity layer manages AI Actor identifiers and signing keys. It comprises:

- `identity/ids.py` — the Identifier dataclass, URI parsing and generation, ULID-based event identifiers, and slug generation. Identifiers follow the `airtrace://<jurisdiction>/<entity-type>/<slug>[#<version-or-instance>]` format and are permanently unique.
- `identity/keys.py` — the KeyService and KeyRecord classes. Ed25519 keys are created, rotated, suspended, revoked, and terminated. Key IDs are permanently unique. Revoked or terminated keys cannot produce valid new events; rotated keys remain verifiable.
- `identity/lifecycle.py` — the Registry and EntityRecord classes. Entities transition through states (proposed, active, suspended, revoked, terminated, archived) with all transitions recorded. Permanent identifiers remain resolvable after termination, revocation, provider closure, model retirement, repository transfer, and organizational restructuring.

The identity layer enforces the permanent identifier invariants: no reassignment, no reuse, permanent resolvability, and binding preservation across execution-context changes.

2.2 Ledger Layer

The ledger layer is the heart of the traceability system. It comprises:

- `ledger/append_only.py` — the AppendOnlyLedger class. Events are append-only; modification and deletion are forbidden. Each event's `previous_event_hash` matches the prior event's `event_hash`, forming a hash chain. The ledger verifies hash-chain integrity, event-hash correctness, and signature validity.
- `ledger/merkle.py` — Merkle root and Merkle proof computation over event sequences. Periodic Merkle roots are anchored to public verification surfaces (e.g., a transparency log or public repository) to provide tamper-evidence beyond the local ledger.
- `signing/ed25519.py` — Ed25519 signature operations and SHA-256 / BLAKE2b hashing.
- `signing/canonical.py` — deterministic JSON canonicalization for signature stability (sorted keys, no whitespace, UTF-8, non-ASCII allowed).

The ledger layer enforces the append-only, tamper-evident, sequence-aware, and cryptographically linked properties. An agent cannot silently alter its canonical history; an administrator cannot silently erase canonical history.

2.3 Governance Layer

The governance layer manages authority chains. It comprises:

- `delegation/broker.py` — the DelegationBroker class. Parent agents delegate bounded scopes of authority to child agents. Delegations are revocable and expirable. Every child agent resolves to a parent delegation; the delegation chain is verifiable end-to-end.
- `authorization/engine.py` — the PolicyEngine, Authorization, and Approval classes. Authorizations record that a principal has authorized an action; approvals are single-use records for high-impact actions. The policy engine evaluates authorization requests against the current policy, enforces scope, and denies fail-closed actions when checks are inconclusive.

The governance layer enforces: every action resolves to an authorization; revoked or expired authorizations produce no new valid actions; policy violations are denied and recorded; approvals are single-use; dual approval is required for the most consequential actions (production deployments, key destruction, schema changes).

2.4 Event Layer

The event layer builds, signs, and records events. It comprises:

- `events/models.py` — the Event, Actor, and ExecutionContext dataclasses. The action vocabulary (75 canonical actions from DISCOVER through TRIGGER_EXTERNAL_EFFECT), visibility tiers (PUBLIC, CONTROLLED, ORGANIZATION_PRIVATE, SEALED), and schema version are defined here.

- `events/collector.py` — the `EventCollector` class. Builds events, signs them with active keys bound to the actor's agent or controller, computes event hashes, appends to the ledger, and returns the typed `Event` object.

The event layer enforces: every event has a unique identifier; every event is hash-chained; every event is signed by an active key bound to the actor; actions and visibility tiers are canonical.

2.5 Adapter Layer

The adapter layer bridges AegisTrace to external systems. It comprises:

- `adapters/filesystem.py` — the `FilesystemAdapter`. Records filesystem operations (`CREATE`, `MODIFY`, `DELETE`) with before/after SHA-256 digests. Provides path-traversal protection.
- `adapters/git.py` — the `GitAdapter`. Wraps Git operations (`commit`, `branch`, `merge`, `tag`) and produces AegisTrace-compatible resource references.
- `adapters/github.py` — the `GitHubEvidenceAdapter`. Produces payloads for the dual-repository evidence pattern: commit attestations, Merkle-root anchors, Merkle proofs, revocation status, and release attestations. Operates locally during the autonomous run; does not push to a live remote.
- `adapters/database.py` — the `DatabaseAdapter`. Wraps SQLite operations (`WRITE`, `DELETE`, `ALTER_SCHEMA`) with before/after digests.
- `adapters/mcp.py` — the `MCPAdapter`. Records MCP (Model Context Protocol) tool invocations.

Adapters do not weaken the invariants. They translate external operations into AegisTrace events with full authority chains.

2.6 Storage Layer

The storage layer persists the ledger, registry, and keys. It comprises:

- `storage/sqlite.py` — the `SQLiteStorage` class. Provides indexed query access to events, registry records, and keys. The ledger is also persisted to JSONL for human readability and external verification.
- JSONL files — the canonical, human-readable, externally-verifiable form of the ledger. JSONL files can be loaded by any AegisTrace instance for verification.

Production deployments should use PostgreSQL-compatible storage with replication; the storage interface is abstract to allow this swap without changing ledger semantics.

3. Data Flow

A typical AegisTrace data flow proceeds as follows. First, a controller registers an agent with the identity layer, creating both a registry record and a signing key bound to that agent. Second, a principal issues an authorization for a specific task and scope via the policy engine; if the planned action requires approval (for example `DEPLOY` or `DESTROY_KEY`), the principal also issues an approval. Third, the agent instance requests to perform an action. Fourth, the policy engine evaluates the request: it verifies that the authorization is active, the action is in scope, and the approval (if required) is valid and not yet used. Fifth, if permitted, the event collector builds an event with the actor, execution context, action, resource, before/after digests, and visibility tier. Sixth, the collector signs the event with the agent's active key, computes the event hash, and appends the event to the ledger. Seventh, the ledger verifies the hash chain and event hash on append. Eighth, periodically, a Merkle root is computed over recent events and anchored to a public verification surface. Ninth, external verifiers can verify the ledger at any time using the `aegistrace.cli.verify` CLI or the Python API.

4. Security Properties

AegisTrace enforces the following security properties, all verified by automated tests in `tests/security/`. Forgery resistance: a signing key bound to one agent cannot forge events under another agent's identity; the collector checks that the signing key's `bound_entity_id` matches the actor's `agent_id` or `controller_id`. Revocation enforcement: a stolen key that has been revoked cannot produce valid new events; the collector refuses to sign with revoked or suspended keys. Hash-chain integrity: any modification, deletion, insertion, or reordering of events is detected by hash-chain verification. Event-hash correctness: any tampering with event content is detected by recomputing and comparing the event hash. Signature verification: any tampering with the signature or the signed content is detected by signature verification. Approval single-use: approvals cannot be reused; a used approval is marked and rejected on subsequent use. Scope enforcement: an action outside the authorized scope is denied and recorded. Fail-closed operation: for high-risk actions (`DEPLOY`, `DESTROY_KEY`, `MODIFY_PRODUCTION`, `ALTER_DATABASE_SCHEMA`), the policy engine denies if any check is inconclusive. Path-traversal protection: the filesystem adapter resolves paths within a base directory and rejects attempts to escape it. Public/private separation: public-tier events contain only non-sensitive fields; sensitive fields are filtered by downstream projection.

5. Privacy Properties

AegisTrace enforces the following privacy properties, all verified by automated tests in `tests/privacy/`. Pseudonymous identifiers: user and principal identifiers are pseudonymous by default (`airace://ca/principal/user-XYZ`); the real identity mapping is sealed. No contact data in events: events never contain real email addresses, phone numbers, or LinkedIn URLs. Visibility tiers: all four visibility tiers (`PUBLIC`, `CONTROLLED`, `ORGANIZATION_PRIVATE`, `SEALED`) are supported. Sealed records: sensitive events use the `SEALED` tier, indicating access only under judicial or regulator-controlled disclosure. User-enumeration prevention: multiple events by the same principal do not leak enumerable information; the pseudonymous identifier is constant.

6. Permanence Properties

AegisTrace enforces the following permanence properties, all verified by automated tests in `tests/permanence/`. Permanent resolvability: a revoked or terminated agent's identifier remains resolvable. Historical signature validity: signatures by a now-revoked key remain verifiable; revocation does not invalidate history. Key rotation preservation: key rotation produces a new key; old events remain verifiable with the rotated key. Identity preservation across model switches: model switch creates a new execution context but preserves the agent's identity. Identity preservation across provider switches: provider switch creates a new execution context but preserves the agent's identity. Archive preservation: archived entities remain resolvable. Ledger permanence after agent termination: after agent termination, the ledger's events remain verifiable.

7. Conformance Properties

AegisTrace enforces the following conformance properties, all verified by automated tests in `tests/conformance/`. Schema conformance: every event conforms to `schemas/event.schema.json`. Canonical action vocabulary: non-canonical actions are rejected. Canonical visibility tiers: the four visibility tiers are the only allowed values. Required invariants: all invariants from `spec/AI-IDP-CORE.md` Section 5 are tested. Append-only: the ledger's public API exposes no mutation methods. Public-tier field separation: PUBLIC-tier events are marked for downstream filtering of sensitive fields. Ledger integrity after many appends: the ledger remains verifiable after 50+ appends.

8. CLI Tools

AegisTrace provides four CLIs. `aegistrace verify` verifies a ledger's hash chain, event hashes, and signatures (with a keys file). `aegistrace audit` produces an audit summary (action counts, agent counts, principal counts, visibility distribution). `aegistrace reconstruct` reconstructs an incident timeline by filtering on task, agent, resource, or time range. `aegistrace admin demo` runs a complete demo scenario that registers entities, creates keys, issues authorizations and approvals, creates a delegation, records events, verifies the ledger, computes the Merkle root, and persists to JSONL.

9. API

The FastAPI server (`aegistrace.api.server`) exposes endpoints for health checks, listing canonical actions and visibility tiers, recording and listing events, verifying the ledger, and resolving entities. The API is intended for integration with agent frameworks, CI/CD systems, and audit dashboards. All endpoints are stateless with respect to the caller; the server holds the in-memory registry, key service, and ledger.

10. Deployment Patterns

AegisTrace supports four deployment patterns. Embedded: the `.aitrace/` directory lives inside the protected directory; suitable for personal projects and small teams. Sidecar: the `.aitrace/` directory lives in a sibling location; suitable for projects that want to keep the trace data separate from the working directory. Central organizational ledger: multiple projects share a central organizational ledger; suitable for enterprises. Federated: multiple organizations federate their ledgers via the federation protocol; suitable for cross-organization collaboration and regulator oversight. For production deployments, the recommended pattern is central organizational ledger with PostgreSQL backend, independent archival replication to a regulator-controlled vault, and Merkle anchoring to a public transparency log.

11. Cryptographic Choices

Signatures use Ed25519 (RFC 8032) via the cryptography library; no custom primitives are implemented. Hashes use SHA-256 for content digests and event hashes; BLAKE2b-256 is available for high-throughput paths. Key management uses an in-memory `KeyService` for the reference implementation; production deployments should use HSM-backed or KMS-backed key storage.

12. Cryptographic Migration

The signing interface is abstract. Ed25519 can be migrated to a post-quantum scheme (e.g., CRYSTALS-Dilithium or SLH-DSA) when standardized and mature. The migration is documented in `spec/PERMANENT_RECORD_PROTOCOL.md`: a new signature scheme is adopted, existing signatures are re-signed with the new scheme, the re-signing is recorded as a new signed event, and the original signatures are preserved.

13. Threat Model

The complete threat model is documented in `threat-model/THREAT_MODEL.md`. The model covers forgery, key compromise, registry tampering, replay, event deletion/insertion/reordering, timestamp manipulation, modified digests, hidden model substitution, adapter bypass, watcher bypass, unauthorized child agents, approval reuse, path traversal, command injection, secret leakage, Git history rewriting, registry tampering, malicious administrator erasure, malicious auditor, and denial of logging. Each threat has a documented mitigation and an automated test.

14. Performance Characteristics

The reference implementation is designed for correctness and clarity, not maximum throughput. On a single core, event recording is approximately 1,000–5,000 events per second (dominated by Ed25519 signing); ledger verification is approximately 10,000–50,000 events per second (hash-chain and signature verification); Merkle root computation is $O(n)$ in event count. Production deployments requiring higher throughput should use a higher-throughput append-only store (e.g., FoundationDB or a purpose-built log-structured store) and batch signature operations. The storage and signing interfaces are abstract to allow this swap.

15. Reproducibility

The reference implementation is reproducible. All dependencies are pinned in `pyproject.toml`. All tests are deterministic (no random seeds; signatures are deterministic Ed25519). The demo scenario (`aegistrace admin demo`) produces a verifiable ledger from scratch. All synthetic benchmark data is generated by reproducible scripts with fixed random seeds, clearly labelled as synthetic.

16. Future Work

Documented future work includes post-quantum signature migration (interface-ready; not implemented), OpenTelemetry runtime exporter (specified; not implemented as a runtime exporter), live federation with an external registry (protocol specified; in-process federation tested), live GitHub remote push (adapter produces ready-to-push payloads; no live push performed), full French body translation (bilingual identity applied; full translation is a separate production task), and external peer review (internal validation performed; external review is a precondition for external publication).

17. Conclusion

The AegisTrace technical architecture provides a complete, functional, tested reference implementation of the AI-IDP standard. It enforces all required invariants, supports all required action coverage, supports all four registry visibility tiers, supports offline operation with reconciliation, and supports federation. The architecture is provider-neutral, jurisdiction-aware, and built on established cryptographic primitives. It is suitable as the basis for a National Standard of Canada, a sectoral certification framework, or an enterprise accountability system.